

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

**ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И
ТЕХНОЛОГИИ“**



**УЕБ СИСТЕМА ЗА НАБЛЮДЕНИЕ НА
ПОКАЗАТЕЛИ В РЕАЛНО ВРЕМЕ: НТТР И
WEBSOCKET СЪРВЪР НА C++ И КЛИЕНТ БЕЗ
ПРИЛОЖНА РАМКА**

КУРСОВ ПРОЕКТ

по дисциплина „Програмиране за Интернет“

Разработил:

Алекс Цветанов, фак. № **[TODO: фак. номер]**

Специалност: Компютърно и софтуерно инженерство, ОКС „Магистър“

Преподавател:

Проф. д-р инж. Огнян Наков

София, **[TODO: година на предаване]**

СЪДЪРЖАНИЕ

УВОД	1
I. АНАЛИЗ НА ПРЕДМЕТНАТА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД	3
1.1. Развитие на динамичното уеб съдържание	3
1.2. Изместени технологии и защо са изместени	3
1.3. Двупосочен обмен между браузър и сървър	4
1.4. Формати за структуриран обмен	4
II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА	5
2.1. Протокол за поток от данни	5
2.2. Приложна рамка на клиента	5
2.3. Съответствие между технологиите от учебната програма и избраните	6
2.4. Формат на обмена	6
2.5. Език и среда на сървъра	7
2.6. Външни зависимости	7
III. ПРОЕКТИРАНЕ НА СИСТЕМАТА	8
3.1. Обща структура	8
3.2. Модел на изпълнение	9
3.3. Договор между сървъра и клиента	10
3.4. Трите транспорта и таблицата с маршрути	11
3.5. Свързване на данни на клиента	11
3.6. Проектни ограничения	12
IV. ПРОГРАМНА РЕАЛИЗАЦИЯ	13
4.1. Организация на изходния текст	13
4.2. Преносим слой за сокети	13
4.3. Разбор на HTTP заявка	14
4.4. Ръкостискане и кадри по WebSocket	14
4.5. Двата формата: изписване и разчитане	17
4.6. Събирач на показатели	18
4.7. Цикъл на сървъра	18

4.8. Клиентска част	19
4.9. Проверки	21
V. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНА ПРОВЕРКА	22
5.1. Какво се проверява	22
5.2. Измервани величини и начин на измерване	22
5.3. Инструмент за натоварване	23
5.4. Условия за валидност на измерването	23
5.5. Апаратна и програмна среда	24
VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ	25
6.1. Закъснение по трите канала	25
6.2. Сравнение на двата формата за обмен	26
6.3. Поведение при нарастващ брой клиенти	27
6.4. Показания на процеса	28
6.5. Обобщение на измерените резултати	28
ЗАКЛЮЧЕНИЕ	29
ЛИТЕРАТУРА	31
ПРИЛОЖЕНИЯ	32

УВОД

Наблюдението на работеща система е задача, при която закъснението има цена. Ако показателят стигне до оператора със закъснение от няколко секунди, картината на екрана описва състояние, което вече не съществува. Класическият уеб модел, при който браузърът пита сървъра през равни интервали и получава пълна страница в отговор, е зле пригоден за тази задача: той харчи заявки, когато нищо не се е променило, и въпреки това изостава, когато промяната настъпи между две запитвания. Двупосочната връзка, при която сървърът изпраща стойност в момента, в който я узнае, решава и двата проблема наведнъж.

Настоящият курсов проект разработва **Pulse**: уеб система за наблюдение на показатели в реално време. Сървърната част е програма на C++20, която сама обслужва HTTP/1.1 [1] за първоначалното зареждане на приложението и WebSocket [2] за потока от стойности след това. Клиентската част е страница без приложна рамка, която стъпва само на браузърните стандарти: обектния модел на документа [3], модела на събитията, структурната графика във формат SVG [4] и разположението с CSS Grid [5]. Отсъствието на рамка е проектно решение, а не пропуск, и е обосновано в Раздел II.

Целта на проекта е да се изгради работеща система за наблюдение в реално време, при която всяка тема от учебното съдържание на дисциплината е представена от действащ елемент на системата, а не от отделен демонстрационен пример. За постигането на целта са формулирани следните **задачи**:

1. преглед на технологиите за обмен на данни между браузър и сървър и на тяхното историческо развитие, включително на изместените от практиката решения;
2. анализ на възможните проектни решения (протокол за поток, формат на данните, начин на изобразяване, наличие или отсъствие на приложна рамка) и обосновка на направения избор;
3. проектиране на архитектурата на системата и на договора между сървъра и клиента;
4. програмна реализация на сървъра на C++20 и на клиента на JavaScript без рамка;
5. изграждане на методика за експериментална проверка на закъснението и на цената на двата формата за обмен;
6. провеждане на измерванията и анализ на получените резултати.

Обхватът на работата е един процес на сървър, който събира показатели и ги раздава, и една страница в браузър, която ги изобразява. Извън обхвата остават разпределеното съхранение на исторически редове, удостоверяването на потребители и разгръщането в производствена среда.

Структура на изложението. Раздел I разглежда предметната област и развитието на технологиите за динамични уеб приложения. Раздел II излага разгледаните алтернативи и обосновава избора между тях. Раздел III представя проектирането на системата, а Раздел IV нейната програмна реализация. Раздел V описва методиката на експеримента, а Раздел VI получените резултати. Заключението обобщава постигнатото и очертава посоките за по-нататъшна работа.

I. АНАЛИЗ НА ПРЕДМЕТНАТА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД

1.1. Развитие на динамичното уеб съдържание

Първото поколение уеб приложения генерира целия документ на сървъра и го изпраща наново при всяко действие на потребителя. Динамиката е на ниво страница: състоянието живее в сървъра, браузърът е програма за изобразяване. Технологиите от този период, сред които Active Server Pages и генерирането на разметка чрез шаблони от страна на сървъра, решават задачата за променливо съдържание, но не и задачата за променливо съдържание *без презареждане*. Всяка промяна струва един пълен обмен и едно ново построяване на документа.

Второто поколение измества част от състоянието в браузъра. Скриптовият език на клиента [6] получава достъп до построеното дърво на документа и може да го променя след като страницата е заредена. Спецификацията на този достъп днес е стандартизирана като обектен модел на документа [3], а разметката и вградените в нея интерфейси като жив стандарт [7]. Едновременно с това се появява асинхронната заявка от скрипт, която по-късно се формализира в общ модел за извличане [8]. От този момент нататък страницата може да поиска данни, без да се разрушава.

1.2. Изместени технологии и защо са изместени

Част от терминологията в учебното съдържание на дисциплината идва от периода, в който разширяването на браузъра ставаше чрез двоични компоненти. Технологията ActiveX позволява изпълнението на native код в контекста на страницата и е налична само в един браузър на една операционна система. Тя решава реален проблем за времето си, защото тогавашният браузър няма нито графика извън растерни изображения, нито мрежов достъп извън заявката за документ. Изместена е по три причини: обвързаност с една платформа, модел на сигурността, който дава на страницата правата на потребителя, и постепенното покриване на същите нужди от самите стандарти. Аналогично, филтрите и преходите, реализирани като собствено разширение на един производител, днес са част от каскадните стилови спецификации и работят във всеки браузър.

Този проект третира избраните технологии *исторически*. Всяка от темите на

дисциплината е реализирана със стандартизирания си наследник, а съответствието между двете е представено в Раздел II. **[TODO: списък на конкретните теми от учебната програма с точното им позоваване, след сверка със силабуса на дисциплината]**

1.3. Двупосочен обмен между браузър и сървър

Периодичното запитване (*polling*) е най-простият начин да се получат нови данни: с всяка заявка клиентът пита дали има промяна. Цената е компромис между излишни заявки при кратък интервал и закъснение при дълъг. Задържаното запитване (*long polling*) намалява едното за сметка на сложност в сървъра, но остава в рамките на модела „заявка, отговор“ на HTTP [9].

Двата съвременни механизма за поток от сървъра към клиента са събитията, изпращани от сървъра (част от разметковия стандарт [7]), и протоколът WebSocket [2]. Първият остава в рамките на HTTP и е еднопосочен. Вторият е отделен протокол, който започва като HTTP заявка за надграждане и след успешно ръкостискане превръща вече установената връзка в двупосочен канал за съобщения. Сравнението между двата е основната проектна алтернатива на този проект и е разгледано в Раздел II.

1.4. Формати за структуриран обмен

За обмена на структурирани данни в мрежата съществуват две широко приети представяния. Разширяемият език за разметка [10] носи схема, пространства от имена и утвърден инструментариум за преобразуване и проверка. Нотацията за обекти на JavaScript [11] е по-кратка, съответства пряко на структурите от данни на клиентския език и се разчита от браузъра без допълнителна библиотека. Проектът реализира *и двата* формата за един и същ поток от показатели, за да може разликата между тях да бъде измерена, а не описана по памет. Показателите, по които двата формата се сравняват, са фиксирани предварително и са три: обем на съобщението по мрежата, време за изписване и време за разбор до пълно дърво. Начинът на измерване на всеки от тях е описан в Раздел V, а получените стойности са в Раздел VI.

II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА

2.1. Протокол за поток от данни

Разгледани са четири варианта за доставяне на нови стойности до браузъра: периодично запитване, задържано запитване, събития, изпращани от сървъра, и WebSocket [2]. Първите два остават в модела „заявка, отговор“ и плащат за всяка стойност цената на пълен обмен от заглавни части. Третият премахва тази цена, но е еднопосочен: клиентът няма как да смени наблюдавания набор от показатели по същия канал и му е нужна отделна заявка за всяко управляващо действие.

Избран е WebSocket, защото задачата изисква и двете посоки: сървърът изпраща стойности, клиентът изпраща команди за смяна на набора, за смяна на честотата и за смяна на формата на обмена. Събитията, изпращани от сървъра, остават реализирани като втори, резервен канал, което позволява двата подхода да бъдат измерени един срещу друг върху един и същ поток от данни, вместо да бъдат сравнявани по литературни твърдения.

Решението е резервният канал да влезе в предадената версия. Реализирани са и трите подхода: WebSocket, събития, изпращани от сървъра, и периодично запитване. Клиентът превключва между тях по време на работа, без презареждане на страницата. Причината е, че без действаща реализация и на трите сравнението в Раздел VI щеше да съпоставя измерена стойност с цитирана, а измерените стойности се получават само от работещ код. Цената е приблизително сто реда в сървъра и три малки модула в клиента.

2.2. Приложна рамка на клиента

Съвременната практика решава задачата за свързване на данни с приложна рамка, която сама следи кои части от документа зависят от кои стойности. Разгледани са два пътя: рамка от познатите днес семейства и клиент без рамка, който работи направо срещу обектния модел на документа [3].

Избран е клиентът без рамка. Мотивът е учебен: цялото съдържание на дисциплината, което се отнася до клиентската част (обектният модел, колекциите, събитията, свързването на данни, структурната графика), при използване на рамка се превръща в извикване на нейни функции и остава скрито. Без рамка същите теми са

видими в изходния текст на проекта. Цената на това решение е ръчно написан слой за свързване на данни, чието проектиране е описано в Раздел III.

2.3. Съответствие между технологиите от учебната програма и избраните

Темите от учебната програма, които сочат към технологии, изместени от практиката, са покрити от стандартизираните им наследници. Съответствието е дадено в таблицата по-долу (Табл. 1), а обосновката за всеки ред следва логиката от Раздел I: избраната технология покрива същата нужда, но е част от стандарт и работи във всеки съвременен браузър.

Таблица 1. Съответствие между технологиите от учебната програма и реализираните в проекта

Тема	Реализирано с	Основание
Двоични компоненти в страницата (ActiveX)	стандартни браузърни интерфейси	еднаква работа във всеки браузър, без права на потребителя за страницата
Генериране на страници на сървъра (ASP)	генериране на HTTP отговор от сървъра на C++20	същата роля, без обвързаност с една платформа
Филтри и преходи като собствено разширение	каскадни стилови преходи и филтри	стандартизирани и хардуерно ускорени
Растерна графика за диаграми	структурна графика SVG [4]	всеки елемент е възел в документа и приема събития
Свързване на данни чрез компонент	собствен слой за свързване върху обектния модел [3]	темата остава видима в изходния текст

2.4. Формат на обмена

Двата разгледани формата, разширяемият език за разметка [10] и нотацията за обекти на JavaScript [11], не се изключват взаимно. Проектът реализира и двата зад един и същ договор за съобщение и позволява формата да се превключва по време на работа. Това решение превръща избора между тях от предположение в измерване и осигурява материала за Раздел VI.

2.5. Език и среда на сървър

Сървърът е на C++20 [12]. Езикът е избран заради прякото управление на паметта и предвидимото време за реакция, което е съществено, когато измерваната величина е закъснението на самата система. Разгледана е и възможността да се използва готов сървър на приложения, при която обработката на протокола отпада; тя е отхвърлена, защото обработката на HTTP/1.1 [1] и на ръкостискането по WebSocket [2] е част от учебното съдържание, а не спомагателна работа.

2.6. Външни зависимости

Разгледани са два пътя: сървърът да стъпи върху готова библиотека за мрежов вход и изход, или да ползва само стандартната библиотека на езика и интерфейса за сокет на операционната система.

Избран е вторият. Хранилището няма нито една задължителна външна зависимост. Мотивът е проверимост: чужд човек трябва да може да изпълни `make -S . -B build` и след това `make --build build` на чиста машина, и изграждането да мине от първия път, без управител на пакети и без изтегляне на нещо от мрежата при конфигуриране. Всичко, което иначе би дошло от библиотека, е написано в проекта и е малко по обем: инструментът за изпълнение на тестовете е сто и деветнадесет реда, измерването на времето стъпва на `std::chrono::steady_clock`, а SHA-1 и Base64 съществуват в изходния текст само защото ръкостискането по WebSocket [2] ги изисква. Свързват се единствено библиотеките, които операционната система така или иначе доставя.

Цената на това решение е, че някои части са по-прости от промишлените си съответствия: анализаторът на разширяемия език за разметка [10] покрива подмножеството, което проектът излъчва, а не целия стандарт. Ограничението е заявено изрично в Раздел IV, за да не бъде прието наум.

III. ПРОЕКТИРАНЕ НА СИСТЕМАТА

3.1. Обща структура

Системата се състои от два процеса и един канал между тях. Сървърният процес чете показатели от операционната система, оформя ги в стойностна снимка, превръща снимката в съобщение и го изпраща по изборния от клиента канал. Клиентският процес е страницата в браузъра: тя приема съобщението, разчита го, прилага го върху състоянието си и обновява само засегнатите части от документа.

Сървърът е разделен на шест части, всяка от които е отделен заглавен файл и отделна единица за превод (Табл. 2). Разделянето следва посоката на данните, а не слоевете на протокола: всяка част приема нещо и връща нещо, и нито една не отваря сокет освен последната.

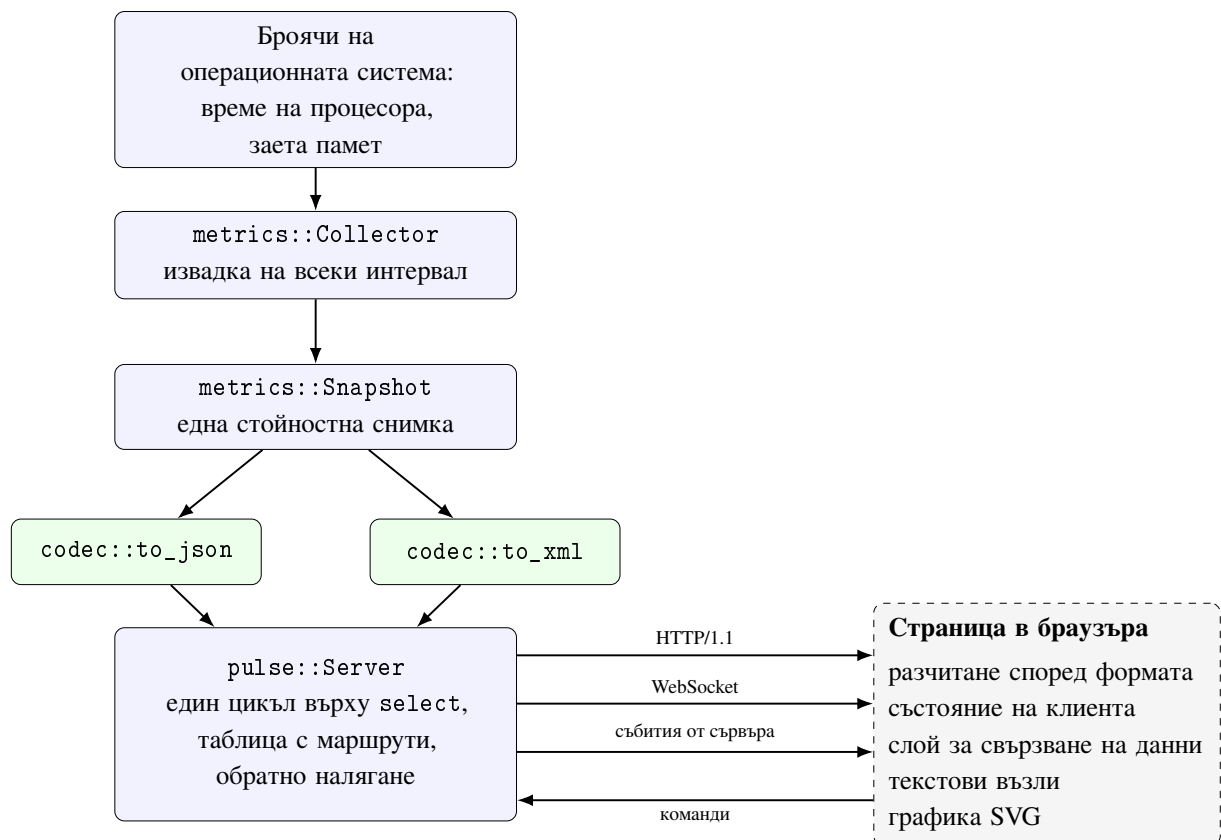
Таблица 2. Части на сървъра и отговорностите им

Част	Приема и връща	Отговорност
net	дескриптор на сокет	еднакъв интерфейс към Winsock и към сокетите на Unix
http	байтове, заявка; отговор, байтове	разбор на заявка, изграждане на отговор, видове съдържание, парчетно кодиране
websocket	байтове, кадър; кадър, байтове	ръкостискане, разбор и изграждане на кадри, събиране на разделени съобщения
metrics	нищо, снимка	четене на броячите на процеса и хистограма на закъснението
codec	снимка, низ; низ, дърво	двата формата за обмен в двете посоки
server	сокет, съобщения	цикълът на събитията, таблицата с маршрути и трите транспорта

Носещото проектно решение е, че *сериализаторът е отделен от източника на показатели*. Един и същ поток от стойности се излъчва в разширяемия език за

разметка [10] или в нотацията за обекти [11], без нито един ред от източника или от цикъла да се променя. Без това разделяне сравнението между двата формата в Раздел VI не би било честно, защото щеше да сравнява две различни реализации, а не два формата.

Общата структура е дадена на схемата по-долу (Фиг. 1). Правоъгълниците със син фон са части на сървъра, тези със зелен фон са двата сериализатора, а пунктираният блок е клиентската страница. Стрелките сочат посоката, в която се движат данните, а надписите върху тях назовават протокола: HTTP/1.1 [1] за страницата, стиловете, скрипта и периодичното запитване; WebSocket [2] за кадрите в двете посоки; и събитията, изпращани от сървъра, за еднопосочния поток. Обратната стрелка носи командите от клиента и съществува само при WebSocket.



Фигура 1. Обща структура на системата и посока на данните

3.2. Модел на изпълнение

Сървърът е *еднонишков*. Един цикъл наблюдава слушащия сокет и всички установени връзки с *select*, обслужва готовите и на всеки интервал взема нова извадка и я разпраща. Няма работни нишки, няма ключалки и няма ред на заключване, за който да се разсъждава, защото всяко споделено състояние (таблицата с връзките, събирачът на

показатели, последната снимка) се пипа от точно една нишка.

Цената на решението е заявена предварително: бавен обработчик би спрял всички клиенти. Тя е приемлива тук, защото нито един обработчик не прави друго освен да оформи низ от стойности, които вече са в паметта, а изходът към сокета е неблокиращ и се доизпраща при следващото минаване на цикъла. Границата, до която този модел издържа, е предмет на измерване в Раздел VI, а не проектна цел.

Всяка връзка носи собствен буфер за вход, буфер за изход и режим: обикновена HTTP връзка, надградена връзка по WebSocket или отворен поток от събития. Режимът се сменя в средата на буфера, а не при нова връзка, което е и причината обработката на буфера да е цикъл, а не единичен избор.

3.3. Договор между сървъра и клиента

Обменът се описва с малък брой видове съобщения (Табл. 3). От сървъра към клиента: стойностна снимка, потвърждение на команда и съобщение за грешка. От клиента към сървъра: смяна на формата, спиране и възобновяване на потока и смяна на интервала. Командите се изпращат само по WebSocket, защото другите два транспорта са еднопосочни; при тях форматът се указва в самата заявка.

Таблица 3. Видове съобщения в договора между сървъра и клиента

Посока	Вид	Полета	Кога
C → K	снимка	seq, ts, uptime, cpu, rss, requests, rps, bytes, ws, sse, p50, p90, p99, histogram	на всеки интервал, до всеки абониран клиент
C → K	потвърждение	type, cmd, value	след приета команда
C → K	грешка	type, message	при неразбрана команда
K → C	format	value: json или xml	при смяна на формата
K → C	subscribe, unsubscribe	няма	при спиране и възобновяване
K → C	interval	value: 50 до 5000	при смяна на честотата

Поредният номер seq съществува по една причина: без него клиентът няма как да отличи изгубено съобщение от съобщение, което просто още не е дошло. Тъй като WebSocket доставя съобщенията в реда на изпращането им по вече установена връзка,

поредният номер служи за откриване на прекъсване и за повторно установяване на състоянието след ново свързване, а не за пренареждане.

Времевият отпечатък `ts` е в микросекунди от началото на епохата и е нужен само за измерването: клиентът изважда от собствения си часовник и получава възрастта на стойността в момента на пристигането ѝ. Двата процеса се изпълняват на една машина, така че въпросът за синхронизация на часовниците отпада.

3.4. Трите транспорта и таблицата с маршрути

Едно и също съдържание се раздава по три различни начина (Табл. 4). Разликата между тях е предметът на измерването и затова и трите са реализирани, а не описани.

Таблица 4. Таблица с маршрути на сървъра

Път	Отговор	Бележка
<code>/ws</code>	101 или 426	надграждане към WebSocket [2]
<code>/api/stream</code>	200, парчетно	събития, изпращани от сървъра
<code>/api/metrics.json</code>	200	последната снимка, за периодично запитване
<code>/api/metrics.xml</code>	200	същата снимка в другия формат
<code>/api/info</code>	200	интервал и списък на крайните точки
всичко останало	200, 403 или 404	файл от папката на клиента

Останалите състояния на отговора са предвидени изрично: 400 при неразбрана заявка, 405 при метод, различен от GET и HEAD, 403 при път, който се опитва да излезе от папката на клиента, и 426, когато заявка към `/ws` пристигне без заглавните части за надграждане. Последното е съществено: то не е грешка в клиента, а заявка, която сървърът не може да изпълни както е написана, и стандартът [9] има точно това състояние за случая.

3.5. Свързване на данни на клиента

Свързването на данни е реализирано като явен списък от тройки: път до стойност в снимката, възел от документа и функция, която записва стойността в този възел. При пристигане на снимка се обхождат само тройките, чиято стойност се различава

от предишната снимка. Този подход е избран вместо пълно преизчисляване на изгледа, защото при няколко обновявания в секунда пълното преизчисляване прави повече работа, отколкото има променени стойности.

Изобразяващият слой е разделен на две. Числовите показатели се изписват като текстови възли и се обновяват през свойството за текстово съдържание. Диаграмите се изграждат като структурна графика [4]: всяка точка от реда е възел в документа, така че върху нея може да се закачи обработчик на събитие и да се приложи стилев преход. Разположението на съставните части на екрана е зададено с CSS Grid [5], а светлата и тъмната тема се различават само по стойностите на потребителските свойства.

Обработката на събития е съсредоточена на едно място чрез *делегиране*: обработчикът се закача на общия родителски възел на диаграмата и разпознава източника на събитието по неговите данни. Решението произтича пряко от факта, че възлите на диаграмата се създават и унищожават при всяко обновяване; обработчик на всеки възел би се закачал и разкачал със същата честота.

3.6. Проектни ограничения

Проектът приема четири ограничения, които стесняват обхвата и са заявени, преди да са измерени резултати.

Първо, състоянието на сървъра се пази в паметта на процеса, което означава, че историята се губи при рестарт. Второ, поддържа се един източник на показатели, а не обединяване на няколко източника. Трето, няма удостоверяване на потребители: всеки клиент, който достигне до крайната точка, получава пълния поток. Ограничението е приемливо, защото системата се изпълнява на локална машина и не се разгръща в производствена среда; това е записано и в обхвата на увода.

Четвърто, интервалът на публикуване е общ за всички клиенти. Команда `interval` от един клиент променя честотата за всички, затова стойността се ограничава между 50 и 5000 милисекунди. Индивидуален интервал за връзка би изисквал отделен график на всяка връзка и не носи нищо на измерването.

IV. ПРОГРАМНА РЕАЛИЗАЦИЯ

4.1. Организация на изходния текст

Хранилището разделя изходния текст на четири части: публичните договори в `include/`, реализацията в `src/`, клиентската част в `web/` и проверките в `tests/` (Табл. 5). Изграждането се управлява от CMake и няма нито една задължителна външна зависимост, по решението от Раздел II.

Таблица 5. Дърво на хранилището

Файл	Редове	Съдържание
<code>include/pulse/*.hpp</code>	478	шестте публични договора
<code>src/net.cpp</code>	179	сокети: Winsock и Unix зад един интерфейс
<code>src/http.cpp</code>	263	разбор на заявка, отговор, видове съдържание
<code>src/websocket.cpp</code>	270	SHA-1, Base64, кадри, събиране на части
<code>src/metrics.cpp</code>	162	бройчи на процеса и хистограма
<code>src/codec.cpp</code>	628	двата формата, изписване и разчитане
<code>src/server.cpp</code>	503	цикъл на събитията, маршрути, транспорти
<code>src/main.cpp</code>	83	двоичният файл на сървъра
<code>src/demo.cpp</code>	537	демонстрация и измервателен инструмент
<code>tests/check.hpp</code>	119	инструмент за изпълнение на тестове
<code>tests/tests.cpp</code>	715	35 случая, 584 проверки
<code>web/index.html</code>	109	един документ
<code>web/app.css</code>	316	разположение, теми, преходи
<code>web/app.js</code>	345	транспорти, свързване, диаграми

4.2. Преносим слой за сокети

Разликите между Winsock и сокетите на семейството Unix са малко на брой, но са навсякъде: типът на дескриптора, стойността за невалиден дескриптор, начинът да се прочете последната грешка, извикването за затваряне и еднократното начално установяване на библиотеката. `pulse::net` ги решава на едно място и останалата част от проекта не съдържа нито едно условно компилиране по операционна система.

Три подробности в този слой не са очевидни. Първо, `FD_SETSIZE` се задава на 1024

преди включването на `winsock2.h`, защото типът `fd_set` се оразмерява от този макрос при компилация, а стойността по подразбиране от 64 не стига за опита с нарастващ брой връзки. Второ, `SO_REUSEADDR` се задава само извън Windows: там значението му е друго и позволява отнемане на действащ слушащ сокет. Трето, при изпращане се подава `MSG_NOSIGNAL`, където съществува, за да не убие сървър сигнал от клиент, който е изчезнал по средата на записа.

4.3. Разбор на HTTP заявка

`http::parse_request` приема буфер и връща едно от три състояния: съобщението е непълно, съобщението е разчетено, или съобщението е повредено. Тази трета стойност е важна: анализаторът не блокира и не чака, а само съобщава, че наличните байтове не стигат, и цикълът го извиква отново, когато дойдат още.

При успех се връща и `consumed`, броят байтове, които съобщението е заело. Оттам идват две свойства наведнъж: постоянната връзка, при която следващата заявка се разчита от остатъка на буфера, и приемането на няколко заявки, изпратени наведнъж. И двете се проверяват от тестове, които разрязват едно съобщение на всички възможни места и очакват състояние „непълно“ за всеки префикс.

Границите се проверяват, преди да се задели памет. Дължината на тялото идва от заглавна част, тоест от ненадежден източник, и се сравнява с горна граница по време на самото ѝ разчитане, а не след него. Същото важи за общия обем на заглавните части.

4.4. Ръкостискане и кадри по WebSocket

Ръкостискането е кратко: стойността на `Sec-WebSocket-Key` се долепя до установената в спецификацията [2] стойност, върху резултата се прилага SHA-1 и се записва в Base64. Проверява се срещу примера от самата спецификация, който е и един от тестовите случаи.

Кадровият анализатор е мястото, където проектът е най-чувствителен към грешки, защото работи изцяло с данни, дошли от мрежата (Листинг 1). Отхвърлят се: запазен бит, поставен без договорено разширение; непознат код на операция; управляващ кадър, който е разделен или е по-дълъг от 125 байта; дължина, записана в по-дълга форма, отколкото е нужно; и дължина над зададената горна граница. Последната проверка се прави *преди* заделянето на памет: разликата между отхвърлен кадър и заделяне, чийто размер е избрал

на срещаният, е точно тази една проверка.

```
1 Decode decode_frame(std::string_view buffer, Frame& out, std::size_t&
  ↳ consumed,
2         std::size_t max_payload) {
3     if (buffer.size() < 2) return Decode::Incomplete;
4     const auto* bytes = reinterpret_cast<const unsigned char*>(buffer.data());
5
6     const bool fin = (bytes[0] & 0x80) != 0;
7     const unsigned reserved = bytes[0] & 0x70;
8     const auto opcode = static_cast<Opcode>(bytes[0] & 0x0F);
9     const bool masked = (bytes[1] & 0x80) != 0;
10    std::uint64_t length = bytes[1] & 0x7F;
11
12    // No extension was negotiated, so a reserved bit set means the peer is
  ↳ speaking a
13    // protocol this server did not agree to.
14    if (reserved != 0) return Decode::Error;
15
16    switch (opcode) {
17        case Opcode::Continuation:
18        case Opcode::Text:
19        case Opcode::Binary:
20        case Opcode::Close:
21        case Opcode::Ping:
22        case Opcode::Pong:
23        break;
24    default:
25        return Decode::Error;
26    }
27    // Control frames carry at most 125 bytes and are never fragmented, section
  ↳ 5.5.
28    if (is_control(opcode) && (length > 125 || !fin)) return Decode::Error;
29
30    std::size_t cursor = 2;
31    if (length == 126) {
32        if (buffer.size() < cursor + 2) return Decode::Incomplete;
33        length = (static_cast<std::uint64_t>(bytes[2]) << 8) | bytes[3];
34        cursor += 2;
```

```

35     if (length < 126) return Decode::Error; // non-minimal length encoding
36 } else if (length == 127) {
37     if (buffer.size() < cursor + 8) return Decode::Incomplete;
38     length = 0;
39     for (int i = 0; i < 8; ++i) {
40         length = (length << 8) | bytes[2 + i];
41     }
42     cursor += 8;
43     if (length <= 0xFFFF) return Decode::Error;
44     if ((length >> 63) != 0) return Decode::Error; // the high bit must be
↪ zero
45 }
46
47 // The declared length comes from the network. Refusing it before reserving
↪ memory is
48 // the difference between a rejected frame and an allocation the peer chose
↪ .
49 if (length > max_payload) return Decode::Error;
50
51 std::array<std::uint8_t, 4> mask{};
52 if (masked) {
53     if (buffer.size() < cursor + 4) return Decode::Incomplete;
54     for (int i = 0; i < 4; ++i) mask[i] = bytes[cursor + i];
55     cursor += 4;
56 }
57
58 const auto payload_size = static_cast<std::size_t>(length);
59 if (buffer.size() < cursor + payload_size) return Decode::Incomplete;
60
61 out.fin = fin;
62 out.opcode = opcode;
63 out.masked = masked;
64 out.payload.assign(buffer.substr(cursor, payload_size));
65 if (masked) {
66     for (std::size_t i = 0; i < payload_size; ++i) {
67         out.payload[i] = static_cast<char>(static_cast<unsigned char>(out.
↪ payload[i]) ^ mask[i % 4]);
68     }
69 }

```

```
70     consumed = cursor + payload_size;
71     return Decode::Ok;
72 }
```

Листинг 1: Разбор на един кадър, `src/websocket.cpp`

Кадрите от клиент към сървър задължително носят маска, а кадър без маска се отхвърля с код 1002 вместо да се обработва. Кадрите в обратната посока никога не носят маска; тестът за това е част от проверката на работещия сървър, а не само на кодека.

Разделените съобщения се събират от `ws::Assembler`. Той пази текущото съобщение и кода на операцията от първата му част, а управляващите кадри, които стандартът позволява да пристигнат между две части, връща отделно и без да ги смесва със събираното съобщение. Отхвърля продължение без начало, ново начало в средата на съобщение и съобщение, което надхвърля зададената горна граница.

4.5. Двата формата: изписване и разчитане

Изписването е право: обхождане на полетата на снимката и допълване на низ. Числата минават през `codec::number` с фиксиран брой знаци след десетичната точка, и двата сериализатора ползват една и съща функция. Това не е разкрасяване: ако единият формат печаташе повече значещи знаци, разликата в обема между двата щеше да отразява форматиране на числа, а не разлика в разметката.

Изписването на текст минава през `escape_json` и `escape_xml`. Тези две функции имат отделни тестове, защото грешка в тях не се вижда в обичайния случай и се проявява само при определени входни данни: знак под `0x20` при нотацията за обекти, и петте предварително определени същности при разметката.

Разчитането е реализирано и за двата формата, защото времето за разбор е една от измерваните величини. И двата анализатора изграждат *пълно дърво*, а не изваждат отделни полета: така сравнението между тях е между еднакъв вид работа, а това е и работата, която браузърът върши при `JSON.parse` и при `DOMParser`. И двата отказват входни данни извън очакваното вместо да ги отгатват: излишно съдържание след документа, непозната същност, незатворен етикет, стойност на признак без кавички, водеща нула в число. Дълбочината на влагане е ограничена, за да не може вход, дошъл от мрежата, да вкара рекурсията в защитната страница на стека.

Анализаторът на разширяемия език за разметка [10] покрива подмножеството, което проектът излъчва: пролог, елементи, признаци, знаково съдържание, петте предварително определени същности, числови препратки, празни елементи и коментари. Пространства от имена, описания на типа на документа и раздели с необработвано съдържание остават извън обхвата. Това е заявено ограничение, не пропуск.

4.6. Събирач на показатели

`metrics::Collector` чете четири неща: време на процесора за този процес, заета памет, брой обслужени заявки и хистограма на времето за обработка на заявка. Първите две идват от операционната система: `GetProcessTimes` и `GetProcessMemoryInfo` под Windows, `getrusage` и `/proc/self/statm` под Linux. Където няма поддържано четене, се връща нула, а не приблизителна стойност: липсваща стойност остава липсваща.

Хистограмата има неподвижни граници на кошовете в микросекунди. Неподвижните граници свеждат добавянето на измерване до сравнение и увеличаване на брояч, което има значение, защото се изпълнява при всяка обслужена заявка. Квантилите се пресмятат с линейна интерполация вътре в коша, който съдържа търсения ранг; последният кош е отворен и за него се връща наблюдаваният максимум, което е и най-честният отговор при отворен кош.

Двата часовника се ползват за различни неща и това е нарочно. `steady_clock` мери всяка продължителност вътре в сървъра, защото стенният часовник може да отскочи назад. `system_clock` дава отпечатъка, който пътува до браузъра, защото браузърът няма достъп до монотонния часовник на сървъра.

4.7. Цикъл на сървъра

Цикълът събира слушащия сокет и всички връзки в два набора за `select`: за четене винаги, за писане само когато има какво да се изпрати. Изчакването се съкращава така, че да не пропусне следващата извадка. След обслужването приключените връзки се премахват от таблицата.

Две решения тук заслужават обяснение.

Обратно налягане. Клиент, който спре да чете, не бива да може да накара сървъра да расте без граница. Изходящият буфер има таван и при надхвърлянето му връзката се затваря. Алтернативата е процесът да бъде убит поради изчерпана памет по избор на един

бавен насрещен.

Смяна на протокола в средата на буфера. Първоначалната реализация обработваше получените байтове според режима на връзката с едно решение: HTTP или WebSocket. Това е грешно и грешката се прояви при първото изпълнение на теста за смяна на формата. Клиент има право да изпрати първия си кадър по WebSocket в същия пакет като заявката за надграждане; режимът се сменя по средата на буфера и остатъкът принадлежи вече на другия протокол. Поправката е обработката да е цикъл, който продължава, докато има напредък, а не единичен избор (Листинг 2). Ако поправката беше направена само в пътя, по който тестът мина, същият пропуск щеше да остане при всяко друго събиране на заявка и кадър в един пакет.

```
1 void process(Connection& connection) {
2     std::size_t before = 0;
3     do {
4         before = connection.incoming.size();
5         switch (connection.mode) {
6             case Mode::Http:      on_http_bytes(connection);      break;
7             case Mode::WebSocket: on_websocket_bytes(connection); break;
8             case Mode::EventStream: connection.incoming.clear();  break;
9         }
10    } while (!connection.incoming.empty() &&
11            connection.incoming.size() != before &&
12            !connection.close_after_write);
13 }
```

Листинг 2: Обработка на получените байтове според режима, src/server.cpp

Потокът от събития се рамкира с парчетно кодиране [1], а не със затваряне на връзката. Дължината на тялото не е известна предварително, а без нея и без парчетно кодиране постоянната връзка не може да се запази.

4.8. Клиентска част

Клиентската част е един документ, един стилев файл и един скрипт, без външни зависимости и без стъпка на изграждане. Скриптът е разделен на четири части: състояние и свързване, разчитане, транспорти и изобразяване.

Свързването е списък от тройки (Листинг 3). Всяка тройка знае едно име от снимката, един възел и една функция за записване. При пристигане на снимка се обхождат само тройките, чиято стойност се е променила спрямо предишната.

```

1  /* The binding layer. Each entry names one path in the snapshot and the way
   ↪ that value
2   * reaches the document. Only entries whose path changed are applied, which
   ↪ is the point
3   * of holding the previous snapshot: at four updates a second, rewriting
   ↪ every node would
4   * do far more work than there are changed values. */
5  const bindings = [
6    { path: 'cpu',      node: 'cpu',      write: (n, v) => (n.textContent = v.
   ↪ toFixed(1)) },
7    { path: 'rss',      node: 'rss',      write: (n, v) => (n.textContent = (v
   ↪ / 1048576).toFixed(1)) },
8    { path: 'requests', node: 'requests', write: (n, v) => (n.textContent = v)
   ↪ },
9    { path: 'rps',      node: 'rps',      write: (n, v) => (n.textContent = v.
   ↪ toFixed(2)) },
10   { path: 'uptime',   node: 'uptime',   write: (n, v) => (n.textContent =
   ↪ Math.round(v)) },
11   { path: 'ws',       node: 'ws',       write: (n, v) => (n.textContent = v)
   ↪ },
12   { path: 'sse',      node: 'sse',      write: (n, v) => (n.textContent = v)
   ↪ },
13   { path: 'p50',      node: 'p50',      write: (n, v) => (n.textContent = v.
   ↪ toFixed(0)) },
14   { path: 'p90',      node: 'p90',      write: (n, v) => (n.textContent = v.
   ↪ toFixed(0)) },
15   { path: 'p99',      node: 'p99',      write: (n, v) => (n.textContent = v.
   ↪ toFixed(0)) },
16 ];
17
18 for (const binding of bindings) binding.node = el(binding.node);
19
20 function applySnapshot(snapshot) {
21   for (const binding of bindings) {
22     const value = snapshot[binding.path];
23     if (value === undefined || value === state.last[binding.path]) continue;
24     binding.write(binding.node, value);
25     flash(binding.node);

```

```

26  }
27  state.last = snapshot;
28
29  state.history.push(snapshot);
30  if (state.history.length > HISTORY) state.history.shift();
31
32  drawSeries();
33  drawHistogram(snapshot.histogram);
34  }

```

Листинг 3: Слой за свързване на данни, `web/app.js`

Двете функции за разчитане връщат един и същ обект, така че нищо след тях не знае кой формат е пристигнал. Времето около самото извикване се измерва с `performance.now` и се показва на екрана: това е цената на формата за клиента, видяна от клиента.

Диаграмата се изгражда от възли на структурната графика [4]. Най-новата извадка стои винаги на десния край, а редът расте наляво, така че хоризонталният мащаб не се променя под вече начертана стойност. Всяка точка е отделен възел с данни за стойността и за поредния номер, а разчитането при посочване минава през един обработчик на общия родител.

Темите са две и се различават само по стойностите на потребителските свойства. Изборът е трипозиционен: по системната настройка, светла и тъмна. При включена настройка за намалено движение всички преходи се съкращават.

4.9. Проверки

Инструментът за изпълнение на тестовете е `tests/check.hpp`, сто и деветнадесет реда: регистрация на случай, сравнение, което отпечатва и двете страни при несъответствие, и ненулев изход, за да го забележи CTest. Наличието на такъв инструмент в хранилището е пряко следствие от решението да няма външни зависимости.

Наборът съдържа 35 случая и 584 проверки. Покрити са: анализаторът на HTTP срещу разрязан и срещу събран вход; кодекът на кадри срещу примерите от спецификацията [2] и срещу повредени кадри; събирането на разделени съобщения; хистограмата; двата сериализатора и двата анализатора; и работещият сървър по истински сокет, по трите транспорта, включително смяната на формата по време на връзката.

V. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНА ПРОВЕРКА

5.1. Какво се проверява

Проверяват се три твърдения, които проектните решения от Раздел II приемат за верни. Първо, че двупосочният канал доставя стойност до клиента по-бързо от периодичното запитване при еднаква честота на обновяване. Второ, че двата формата за обмен се различават по обем на съобщението и по време за разбор. Трето, че системата запазва работоспособност при нарастващ брой едновременни клиенти до определена граница, която експериментът трябва да намери, а не да предположи.

Всяко от трите твърдения е записано тук *преди* провеждането на измерването. Целта е една погрешна прогноза да остане видима и да бъде отчетена като погрешна, вместо да бъде пренаписана след получаването на резултата.

5.2. Измервани величини и начин на измерване

Закъснение. Измерва се като *възраст на стойността при пристигане*: от момента, в който събирачът произведе извадка, до момента, в който клиентът разчете съобщението. Всяко съобщение носи времеви отпечатък от сървъра в поле `ts`, а клиентът изважда този отпечатък от собствения си часовник. И двете страни се изпълняват на една машина, така че въпросът за синхронизация на часовниците отпада.

Източникът на време от двете страни е `std::chrono::system_clock`, приведен до микросекунди. Изборът на стенен, а не на монотонен часовник е наложен от задачата: двата края са различни обекти и монотонният часовник няма обща начална точка между тях. Всички продължителности, измервани изцяло вътре в един процес, ползват `std::chrono::steady_clock`. Разделителната способност на `system_clock` е измерена на използваната машина, а не приета наум: най-малката наблюдавана ненулева стъпка между две последователни четения е 100 ns, тоест поне с три порядъка по-малка от измерваните закъснения. Самото четене на часовника се извършва веднъж на съобщение и затова не влиза в измерваната величина.

Обем на съобщението. Измерва се като брой байтове по мрежата за един и същ пакет със стойности: при WebSocket [2] байтовете на кадъра, включително заглавната му част; при събитията, изпращани от сървъра, байтовете на парчето заедно с рамката на събитието; при периодичното запитване байтовете на заявката и на пълния отговор,

заглавните части включително. Мери се доставеното по мрежата, а не дължината на полезния товар, защото именно доставеното се плаща.

Време за изписване и за разбор. Измерва се вътре в процеса, около самото извикване, с монотонния часовник. И двата анализатора изграждат пълно дърво, за да е сравнението между еднакъв вид работа, а не между изваждане на едно поле и построяване на документ.

Пропускателна способност при нарастващ брой клиенти. Измерва се като брой доставени съобщения за фиксиран прозорец при 1, 8, 32 и 128 едновременни връзки по WebSocket, съпоставен с очаквания брой, който е произведение на броя клиенти и броя интервали в прозореца.

5.3. Инструмент за натоварване

Натоварването и измерването се извършват от pulse-demo, изграден от същото хранилище. Той стартира сървър в отделна нишка в същия процес, свързва се към него по локалния интерфейс като обикновен клиент и говори същите три протокола. Клиентската страна не ползва браузър: браузърният клиент е проверен функционално, но не участва в числата, защото планировчикът на страницата и честотата на опресняване на екрана биха се смесили с измерваната величина.

Това решение има две последствия, които трябва да бъдат заявени. Първо, измерващият клиент и сървърът делят един процес, затова показанието за време на процесора в края на опита включва и работата на измерващия клиент; то се отчита като показание на процеса, а не като цена на сървър. Второ, обменът минава през стека на локалния интерфейс, тоест измерените закъснения не съдържат мрежово време и представляват долна граница за реална мрежа.

Че самият инструмент не е тясното място, се установява по следния начин: при 128 връзки броят доставени съобщения се сравнява с очаквания. Ако измерващият клиент изоставаше, доставените съобщения биха били по-малко от очакваните, а закъснението би нараствало с времето вътре в прозореца. И двете се отчитат в Раздел VI.

5.4. Условия за валидност на измерването

Условията са фиксирани предварително и не са променяни, след като са видени резултати.

1. Интервалът на публикуване е 100 милисекунди за всички опити.
2. Изписването и разборът се измерват в 5 кръга по 10 000 повторения. Отчита се средното на *средния* кръг, а най-ниският и най-високият кръг се отчитат като мярка за разсейване. Един кръг сам по себе си се различава от друг с множител, близък до две, затова едно число от един кръг не би било възпроизводимо.
3. За всеки транспорт се събират 65 съобщения, от които първите 5 се отхвърлят. Отхвърлените носят еднократните разходи: установяване на връзката, първата извадка и страниците, които разпределителят на памет още не е докоснал.
4. Опитът с нарастващ брой клиенти ползва прозорец от 3 секунди за всяка стойност на броя клиенти.
5. Отчитат се средна стойност, медиана и деветдесет и пет проценти, а не само средна стойност.
6. По време на опита на машината не се изпълнява друго забележимо натоварване, а браузърът е затворен.

Целият опит се възпроизвежда с една команда, `pulse-demo --bench`, а изведеният от нея отчет е приложен без редакция в Приложение Г.

5.5. Апаратна и програмна среда

Средата на измерването е част от резултата и е описана изцяло, за да може опитът да бъде повторен (Табл. 6).

Таблица 6. Апаратна и програмна среда на измерването

Елемент	Стойност
Процесор	AMD Ryzen 5 3600, 6 ядра, 12 логически процесора
Оперативна памет	15,9 GiB
Операционна система	Windows 11 Pro N, версия 10.0.26200
Компилятор	g++ 15.2.0, MinGW-w64, x86_64-posix-seh
Стандарт на езика	C++20, без разширения на компилатора
Ниво на оптимизация	CMAKE_BUILD_TYPE=Release
Система за изграждане	CMake 4.3.2, Ninja 1.13.2
Мрежов път	локален интерфейс, 127.0.0.1
Интервал на публикуване	100 ms
Браузър	не участва в измерването, вж. подраздел 5.3

VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ

Всички числа в този раздел са получени от едно изпълнение на `pulse-demo --bench` върху средата, описана в Раздел V (Табл. 6). Суровият отчет от същото изпълнение е приложен без редакция в Приложение Г.

6.1. Закъснение по трите канала

Измерена е възрастта на стойността при пристигане при доставка по WebSocket [2], по събития, изпращани от сървър, и по периодично запитване, при еднакъв интервал на публикуване от 100 ms (Табл. 7). За всеки канал са събрани 65 съобщения, от които първите 5 са отхвърлени по условията от Раздел V.

Таблица 7. Възраст на стойността при пристигане и обем на едно обновяване, по вид на канала

Канал	Брой	Средно ms	Медиана ms	95-и проц. ms	Байта на обн.
WebSocket	60	0,47	0,15	0,76	373,7
Събития от сървър	60	0,66	0,16	3,02	410,7
Периодично запитване	60	42,20	46,59	92,00	561,3

Прогнозата от Раздел V се потвърждава, но не по причината, която би се очаквала. Разликата не е в скоростта на самия пренос: медианата на двата потокови канала е 0,15 и 0,16 ms, тоест те са неразличими помежду си. Разликата е в това, че при периодично запитване стойността вече е *остаряла*, когато клиентът я поиска. Сървърът взема извадка на всеки 100 ms; запитване, което пристига в произволен момент между две извадки, получава стойност на средна възраст половин интервал. Измерената средна стойност от 42,20 ms при интервал от 100 ms съответства точно на това очакване, а деветдесет и петият процентил от 92,00 ms е близо до пълния интервал, което е и най-лошият възможен случай.

Заключението е по-точно от първоначалното твърдение: периодичното запитване не е бавно, то е *ненавременно*. Съкращаването на интервала на запитване не решава проблема, а го заменя с друг, защото всяко запитване струва пълен обмен от заглавни части: 561,3 байта срещу 373,7 при WebSocket за едно и също съдържание, тоест с 50% повече.

Двата потокови канала се различават осезаемо само по деветдесет и петия

процентил: 0,76 ms при WebSocket срещу 3,02 ms при събитията от сървъра. Разликата идва от рамкирането: събитието се предава с парчетно кодиране [1] и рамка от текстови редове, докато кадърът по WebSocket носи двоична заглавна част от два до четири байта. Това личи и в обема: 410,7 срещу 373,7 байта на обновяване.

6.2. Сравнение на двата формата за обмен

За една и съща стойностна снимка са измерени обемът на съобщението, времето за изписване и времето за разбор при сериализация в разширяемия език за разметка [10] и в нотацията за обекти [11] (Табл. 8). Измерването е в 5 кръга по 10 000 повторения; отчита се средният кръг, а в квадратни скоби е разсейването между най-ниския и най-високия кръг.

Таблица 8. Обем, време за изписване и време за разбор по формат на обмена

Формат	Байта	Изпис.	[мин, макс]	Разбор	[мин, макс]	Възли
		μs	μs	μs	μs	
JSON	386	6,803	[5,35; 7,45]	17,268	[14,89; 19,73]	42
XML	566	7,522	[7,42; 8,03]	8,191	[7,42; 9,20]	33

Твърдението за обема се потвърждава. XML е с 46,6% по-обемен от JSON за едно и също съдържание. Разликата идва изцяло от разметката: числата минават през една и съща функция за изписване в двата случая, така че форматирането им не участва в резултата.

Твърдението за времето за разбор се потвърждава по посока, обратна на очакваната. В тази реализация XML се разчита за 8,191 μs срещу 17,268 μs за JSON, тоест за 0,47 от времето. Причината не е в синтаксиса, а в *формата на документа*. Хистограмата се записва в JSON като масив от девет обекта с по две полета, което дава 42 възела в дървото. В XML същата хистограма е девет елемента с признак и текстово съдържание, което дава 33 възела. Изграждането на възел, а не разчитането на знаци, е основната цена и при двата анализатора.

Този резултат не бива да се обобщава до твърдението, че XML се разчита по-бързо от JSON. Правилното заключение е по-тясно и по-полезно: *при сравними анализатори цената на разбора се определя главно от броя на изградените възли, а броят възли зависи от това как е моделиран документът, а не от избрания синтаксис*. Същите

данни, записани в JSON като масив от двойки вместо като масив от обекти, биха дали друго съотношение.

Времето за изписване е практически еднакво: 6,803 срещу 7,522 μ s, като разсейването между кръговете при JSON достига същия порядък като самата разлика. От измерването не следва, че единият формат се изписва по-бързо от другия.

Компресията на транспортно ниво не е измервана. Тя би променила съотношението по обем, тъй като повтарящите се имена на етикети в XML се свиват добре, но добавянето на компресор изисква външна библиотека и противоречи на решението за отсъствие на зависимости от Раздел II. Стойностите в Таблица 8 са за некомпесиран обмен и трябва да се четат така.

6.3. Поведение при нарастващ брой клиенти

Измерени са броят доставени съобщения и закъснението при 1, 8, 32 и 128 едновременни връзки по WebSocket, за прозорец от 3 секунди при интервал на публикуване 100 ms (Табл. 9). Очакваният брой е произведение на броя клиенти и броя интервали в прозореца.

Таблица 9. Доставени съобщения и закъснение при нарастващ брой едновременни връзки

Клиенти	Доставени	Очаквани	Запазени %	Средно ms	95-и проц. ms
1	30	30	100,0	0,30	0,46
8	240	240	100,0	0,29	0,55
32	960	960	100,0	0,87	2,09
128	3 840	3 840	100,0	2,04	4,89

Границата не е достигната. До 128 едновременни връзки не е изпуснато нито едно съобщение: доставените са точно колкото очакваните и при четирите стойности. Закъснението нараства, но остава далеч под интервала на публикуване: от 0,30 ms при една връзка до 2,04 ms при 128, при интервал от 100 ms. Нарастването е очаквано, защото при 128 връзки един цикъл оформя и записва 128 кадъра, преди да се върне към чакане.

Понеже доставените съобщения съвпадат с очакваните и при най-високата стойност, измерваният клиент не е изоставал и не е тясното място по критерия от Раздел V. Тоест измереното е поведение на сървъра, а не на инструмента.

Твърдението от Раздел V, че съществува граница, която опитът трябва да намери, *не е проверено докрай*: опитът показва само, че границата е над 128 връзки. Тя не е потърсена

по-нагоре по две причини. Първо, измерващият клиент дели процес и машина със сървър, така че при по-голям брой връзки двете страни щяха да се борят за едни и същи ядра и резултатът щеше да описва машината, а не сървъра. Второ, наборът `fd_set` е оразмерен на 1024 при компилация, което поставя таван на реализацията, независим от натоварването. И двете са ограничения на измерването, а не резултати от него.

6.4. Показания на процеса

В края на опита сървърът отчита 6,30 MiB заета памет и време за обработка на една заявка с медиана 25,2 μ s, деветдесети перцентил 45,4 μ s и деветдесет и девети перцентил 49,9 μ s. Отчетените 236 заявки и 1 998 894 изпратени байта са за целия опит.

Показанието за време на процесора не се отчита като резултат: измерващият клиент се изпълнява в същия процес и работата му влиза в същия брояч, както е заявено в Раздел V.

6.5. Обобщение на измерените резултати

1. **Двупосочният канал доставя стойността по-навреме от периодичното запитване: потвърдено.** Медиана 0,15 ms срещу 46,59 ms, при 33% по-малък обем на обновяването. Причината обаче не е скорост на преноса, а възрастта на стойността при запитване; формулировката в Раздел V беше неточна и е поправена тук.
2. **Двата формата се различават по обем и по време за разбор: потвърдено, но посоката на втората разлика е обратна на очакваната.** XML е с 46,6% по-обемен и се разчита за 0,47 от времето на JSON, защото дървото му има 33 възела срещу 42. Цената на разбора следва броя възли, а не синтаксиса.
3. **Съществува граница по брой едновременни клиенти: непроверено.** До 128 връзки системата не изпуска нищо. Границата е над тази стойност и намирането ѝ изисква измерващ клиент на отделна машина.

Ограничения на самото измерване. Обменът минава през локалния интерфейс, тоест измерените закъснения са долна граница за реална мрежа. Измерващият клиент дели процес със сървър. Компресията на транспортно ниво не е измервана. Едно изпълнение на опита е недостатъчно, за да се говори за разсейване между изпълнения; отчетеното разсейване е вътре в едно изпълнение.

ЗАКЛЮЧЕНИЕ

В рамките на курсовия проект е разработена система за наблюдение на показатели в реално време, изградена от сървър на C++20 [12], който сам обслужва HTTP/1.1 [1] и WebSocket [2], и от клиент в брауъра без приложна рамка. Клиентската част стъпва пряко върху обектния модел на документа [3], модела на събитията, структурната графика [4] и разположението с CSS Grid [5], а обменът е реализиран в два формата [10, 11], за да може разликата между тях да бъде измерена.

Предимства на решението. Отсъствието на приложна рамка прави всяка стъпка от обработката видима в изходния текст, а сървър, който сам обработва протокола, дава пълен контрол над момента на изпращане, което е предпоставка за измерването на закъснението. Разделянето на сериализатора от източника на показатели позволява смяна на формата по време на работа, без промяна в останалата част от системата.

Недостатъци и ограничения. Слой за свързване на данни е написан ръчно и изисква явно описание на всяка зависимост, което при разрастване на интерфейса става източник на пропуски. Състоянието се пази в паметта на процеса и се губи при рестарт. Няма удостоверяване на потребители. Тези ограничения са заявени в Раздел III преди измерването, а не открити след него.

Инженерна трактовка на резултатите. Измерването в Раздел VI дава два извода, които се пренасят извън този проект.

Първият се отнася до избора на канал. Периодичното запитване не изостава защото пренася бавно, а защото връща стойност, която вече е остаряла: при интервал на публикуване от 100 ms средната ѝ възраст е половин интервал, а най-лошата е цял. Затова съкращаването на интервала на запитване не е решение, а размяна на едно затруднение за друго, тъй като всяко запитване струва пълен обмен от заглавни части, в случая с 50% повече байтове за едно и също съдържание. За задача, при която стойността има възраст, потоковият канал е правилният избор, а между двата потокови канала разликата е малка: измерените медиани са неразличими, а WebSocket [2] се откроява само по по-стегнатото рамкиране и по това, че носи и обратната посока.

Вторият се отнася до избора на формат и е обратен на очакваното. Разширяемият език за разметка [10] се оказа с 46,6% по-обемен, но се разчете за по-малко от половината от времето на нотацията за обекти [11], защото дървото му съдържа по-малко възли за

същите данни. Изводът не е, че единият формат е по-бърз от другия, а че цената на разбора следва *броя на изградените възли*, тоест начина, по който документът е моделиран, а не избрания синтаксис. При проектиране на обмен това означава, че формата на съобщението заслужава повече внимание от спора кой от двата формата да се ползва.

Двата извода имат обща поука за метода: и двата опровергават формулировка, записана преди опита. Ако твърденията не бяха записани предварително, разликата между очакваното и измереното нямаше да остане видима.

Насоки за по-нататъшна работа. Естествените продължения са съхранение на исторически редове извън паметта на процеса, обединяване на няколко източника на показатели, удостоверяване на потребители и разширяване на слоя за свързване на данни с автоматично проследяване на зависимостите.

ЛИТЕРАТУРА

- [1] R. Fielding, M. Nottingham, and J. Reschke, “HTTP/1.1,” Request for Comments RFC 9112, Internet Engineering Task Force, June 2022.
- [2] I. Fette and A. Melnikov, “The WebSocket protocol,” Request for Comments RFC 6455, Internet Engineering Task Force, Dec. 2011.
- [3] WHATWG, “DOM standard.” Living Standard. **[TODO: дата на достъп]**.
- [4] W3C SVG Working Group, “Scalable vector graphics (SVG) 2.” W3C Specification. **[TODO: статус на документа и дата на достъп]**.
- [5] W3C CSS Working Group, “CSS grid layout module level 1.” W3C Specification. **[TODO: статус на документа и дата на достъп]**.
- [6] Ecma International, “ECMA-262: ECMAScript language specification.” Standard. **[TODO: издание и година на цитираното издание]**.
- [7] WHATWG, “HTML standard.” Living Standard. **[TODO: дата на достъп]**.
- [8] WHATWG, “Fetch standard.” Living Standard. **[TODO: дата на достъп]**.
- [9] R. Fielding, M. Nottingham, and J. Reschke, “HTTP semantics,” Request for Comments RFC 9110, Internet Engineering Task Force, June 2022.
- [10] W3C XML Core Working Group, “Extensible markup language (XML) 1.0 (fifth edition).” W3C Recommendation, Nov. 2008.
- [11] T. Bray, “The JavaScript object notation (JSON) data interchange format,” Request for Comments RFC 8259, Internet Engineering Task Force, Dec. 2017.
- [12] ISO/IEC, “ISO/IEC 14882:2020, programming languages, C++.” International Standard, 2020.

ПРИЛОЖЕНИЯ

Приложение А. Формат на съобщенията

Пълното описание на видовете съобщения от договора между сървъра и клиента е дадено в Раздел III (Табл. 3). Тук е добавено значението на всяко поле от стойностната снимка (Табл. 10), следвано от по един действителен пример във всеки от двата формата на обмена [10, 11]. Примерите са снети от работещ сървър и не са редактирани.

Таблица 10. Полета на стойностната снимка

Поле	Вид	Значение
seq	цяло	пореден номер на извадката, започва от 1
ts	цяло	време на извадката, микросекунди от началото на епохата
uptime	реално	секунди от стартирането на сървъра
cpu	реално	процент от едно ядро за последния интервал
rss	цяло	заета памет на процеса в байтове
requests	цяло	общ брой обслужени заявки
rps	реално	заявки за секунда за последния интервал
bytes	цяло	общо изпратени байтове
ws	цяло	брой връзки по WebSocket
sse	цяло	брой отворени потока от събития
p50, p90, p99	реално	време за обработка на заявка в микросекунди
histogram	списък	девет коша: горна граница и брой попадения

```
1 {"seq":12,"ts":1787093650413132,"uptime":2.766,"cpu":0.00,"rss":5611520,  
2  "requests":0,"rps":0.00,"bytes":0,"ws":0,"sse":0,  
3  "p50":0.00,"p90":0.00,"p99":0.00,  
4  "histogram":[{"le":50,"count":0}, {"le":100,"count":0}, {"le":200,"count":0},  
5  {"le":500,"count":0}, {"le":1000,"count":0}, {"le":2000,"count":0},  
6  {"le":5000,"count":0}, {"le":10000,"count":0}, {"le":null,"count":0}]}
```

Листинг 4: Стойностна снимка в нотацията за обекти

```
1 <?xml version="1.0" encoding="UTF-8"?><snapshot><seq>13</seq>  
2 <ts>1787093650648926</ts><uptime>3.002</uptime><cpu>0.00</cpu>  
3 <rss>5664768</rss><requests>1</requests><rps>4.24</rps><bytes>473</bytes>  
4 <ws>0</ws><sse>0</sse><p50>25.00</p50><p90>45.00</p90><p99>49.50</p99>
```

```

5 <histogram><bucket le="50">1</bucket><bucket le="100">0</bucket>
6 <bucket le="200">0</bucket><bucket le="500">0</bucket>
7 <bucket le="1000">0</bucket><bucket le="2000">0</bucket>
8 <bucket le="5000">0</bucket><bucket le="10000">0</bucket>
9 <bucket le="inf">0</bucket></histogram></snapshot>

```

Листинг 5: Същата снимка в разширяемия език за разметка

По мрежата и двата документа се предават на един ред. Тук са пренесени на няколко реда само за да се поберат в полето на страницата; байтовете за нов ред не са част от съобщението, а обемите в Раздел VI са за действително предаденото.

Команда от клиента към сървъра е обект с поле `cmd` и незадължително поле `value`, а отговорът е потвърждение или съобщение за грешка:

```

1 клиент -> сървър {"cmd":"format","value":"xml"}
2 сървър -> клиент {"type":"ack","cmd":"format","value":"xml"}
3 сървър -> клиент {"type":"error","message":"unknown command"}

```

Листинг 6: Команда, потвърждение и грешка

Приложение Б. Избрани части от изходния текст

Разборът на един кадър по WebSocket [2] е даден в Раздел IV (Листинг 1), а слоят за свързване на данни на клиента там (Листинг 3). Тук следват двата сериализатора, които реализират един и същ вътрешен договор, и събирането на разделени съобщения.

```

1 std::string to_json(const metrics::Snapshot& s) {
2     std::string out;
3     out.reserve(768);
4     out += "{ \"seq\": ";
5     out += std::to_string(s.sequence);
6     out += ", \"ts\": ";
7     out += std::to_string(s.wall_microseconds);
8     out += ", \"uptime\": ";
9     out += number(s.uptime_seconds, 3);
10    out += ", \"cpu\": ";
11    out += number(s.cpu_percent, 2);
12    out += ", \"rss\": ";
13    out += std::to_string(s.rss_bytes);
14    out += ", \"requests\": ";

```

```

15 out += std::to_string(s.requests_total);
16 out += ", \"rps\": ";
17 out += number(s.requests_per_second, 2);
18 out += ", \"bytes\": ";
19 out += std::to_string(s.bytes_sent);
20 out += ", \"ws\": ";
21 out += std::to_string(s.websocket_clients);
22 out += ", \"sse\": ";
23 out += std::to_string(s.sse_clients);
24 out += ", \"p50\": ";
25 out += number(s.latency_p50, 2);
26 out += ", \"p90\": ";
27 out += number(s.latency_p90, 2);
28 out += ", \"p99\": ";
29 out += number(s.latency_p99, 2);
30 out += ", \"histogram\": [ ";
31 const auto& bounds = metrics::Histogram::bounds();
32 for (std::size_t i = 0; i < metrics::Histogram::kBucketCount; ++i) {
33     if (i > 0) out += ', ';
34     out += "{ \"le\": ";
35     if (i < bounds.size()) {
36         out += number(bounds[i], 0);
37     } else {
38         out += "null"; // the open ended bucket has no upper bound
39     }
40     out += ", \"count\": ";
41     out += std::to_string(s.histogram.buckets()[i]);
42     out += '}, ';
43 }
44 out += " ] ";
45 return out;
46 }
47
48 std::string to_xml(const metrics::Snapshot& s) {
49     std::string out;
50     out.reserve(1024);
51     out += "<?xml version=\"1.0\" encoding=\"UTF-8\"?>";
52     out += "<snapshot><seq>";
53     out += std::to_string(s.sequence);

```

```

54 out += "</seq><ts>";
55 out += std::to_string(s.wall_microseconds);
56 out += "</ts><uptime>";
57 out += number(s.uptime_seconds, 3);
58 out += "</uptime><cpu>";
59 out += number(s.cpu_percent, 2);
60 out += "</cpu><rss>";
61 out += std::to_string(s.rss_bytes);
62 out += "</rss><requests>";
63 out += std::to_string(s.requests_total);
64 out += "</requests><rps>";
65 out += number(s.requests_per_second, 2);
66 out += "</rps><bytes>";
67 out += std::to_string(s.bytes_sent);
68 out += "</bytes><ws>";
69 out += std::to_string(s.websocket_clients);
70 out += "</ws><sse>";
71 out += std::to_string(s.sse_clients);
72 out += "</sse><p50>";
73 out += number(s.latency_p50, 2);
74 out += "</p50><p90>";
75 out += number(s.latency_p90, 2);
76 out += "</p90><p99>";
77 out += number(s.latency_p99, 2);
78 out += "</p99><histogram>";
79 const auto& bounds = metrics::Histogram::bounds();
80 for (std::size_t i = 0; i < metrics::Histogram::kBucketCount; ++i) {
81     out += "<bucket le=\"";
82     out += i < bounds.size() ? number(bounds[i], 0) : std::string("inf");
83     out += "\">";
84     out += std::to_string(s.histogram.buckets()[i]);
85     out += "</bucket>";
86 }
87 out += "</histogram></snapshot>";
88 return out;
89 }

```

Листинг 7: Двата сериализатора, src/codecs.cpp

```

1  Assembler::Event Assembler::feed(const Frame& frame) {
2      if (is_control(frame.opcode)) {
3          opcode_ = frame.opcode;
4          payload_ = frame.payload;
5          return Event::Control;
6      }
7
8      if (frame.opcode == Opcode::Continuation) {
9          if (!in_message_) return Event::Protocol; // continuation without a
↪ start frame
10     } else {
11         if (in_message_) return Event::Protocol; // a new data frame inside a
↪ message
12         in_message_ = true;
13         pending_.clear();
14         pending_opcode_ = frame.opcode;
15     }
16
17     if (pending_.size() + frame.payload.size() > max_message_) {
18         in_message_ = false;
19         pending_.clear();
20         return Event::Protocol;
21     }
22     pending_ += frame.payload;
23
24     if (!frame.fin) return Event::NeedMore;
25
26     payload_.swap(pending_);
27     pending_.clear();
28     opcode_ = pending_opcode_;
29     in_message_ = false;
30     return Event::Message;
31 }

```

Листинг 8: Събиране на разделени съобщения, src/websocket.cpp

Приложение В. Екранни форми

Тук се включват екранните снимки на готовия интерфейс: изгледът с числовите показатели, изгледът с диаграмите и състоянието при прекъсната връзка, в светла и в тъмна тема. **[TODO: екранни снимки с надписи под тях]**

Приложение Г. Отчет от измерванията

По-долу е изведеното от `pulse-demo --bench` върху средата от Раздел V (Табл. 6), без редакция. Всички числа в Раздел VI произхождат от този отчет.

```
Pulse: live metrics over HTTP/1.1, WebSocket and server-sent events
Server listening on port 57270, publishing every 100 ms

JSON against XML, same snapshot, 5 rounds of 10000 repetitions
Times are the median round, with the lowest and highest round in brackets.

-----
format    bytes  encode us [low, high]      parse us [low, high]      nodes
-----
JSON      386    6.803 [5.35, 7.45]      17.268 [14.89, 19.73]     42
XML       566    7.522 [7.42, 8.03]      8.191 [7.42, 9.20]       33
-----

XML is 46.6% larger, encodes 1.11x and parses 0.47x the JSON time.

Collecting 65 updates over each transport, this takes about 20 seconds.

Transports, 100 ms publish interval, one client, loopback

-----
transport  updates  mean ms   p50 ms   p95 ms   bytes/update
-----
WebSocket      60      0.47     0.15     0.76     373.7
SSE             60      0.66     0.16     3.02     410.7
Polling        60     42.20    46.59    92.00     561.3
-----

Latency is the age of the value on arrival: the client clock minus the
timestamp the server wrote when it sampled. Both ends share one machine.

Concurrent WebSocket clients, 3000 ms window, 100 ms publish interval

-----
clients  delivered  expected  kept %   mean ms   p95 ms
-----
      1         30         30    100.0     0.30     0.46
      8        240        240    100.0     0.29     0.55
     32        960        960    100.0     0.87     2.09
    128       3840       3840    100.0     2.04     4.89
```

```
-----  
The expected count is clients times windows per interval. Both the server  
and the measuring clients run on the same machine and share its cores.  
  
Server state after the run  
-----  
requests served    236  
bytes sent         1998894  
resident memory    6.30 MiB  
process cpu        67.82 % of one core in the last interval  
request handling   p50 25.2 us, p90 45.4 us, p99 49.9 us  
-----
```

Листинг 9: Отчет от измерването, `measurements/report.txt`